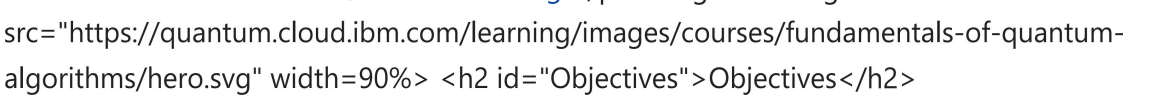
<h1 id="Quantum-Gates-Homework">Quantum Gates Homework</h1><h2 id="PLEASE-READ-ALL-SECTIONS">PLEASE READ ALL SECTIONS</h2><h2 id="Overview">Overview</h2> In this tutorial, you will learn to build quantum circuits using Qiskit. Qiskit is an open-source Python library primarily written in Rust. With its huge community, Qiskit has excellent performance and learning resources. You can learn more about Qiskit on IBM Quantum Learning. 

<h2 id="Objectives">Objectives</h2>

- Learn how to use the Qiskit API
- Understand how to create and manipulate statevectors
- Apply matrix and tensor math to design quantum gates
- Build quantum circuits

<h2 id="Grading">Grading</h2> You are expected to write your own code. It is acceptable to find and use code from the Qiskit API and IBM Quantum Learning webpages, but **it is strictly prohibited to use code obtained from other sources**, especially other students. You are also expected to write descriptive comments explaining what your code is trying to accomplish and why. **DO NOT MODIFY PRE-EXISTING CODE! ONLY WRITE CODE WHERE SPECIFIED!** **NOT FOLLOWING INSTRUCTIONS MAKES GRADING MORE DIFFICULT FOR THE GRADERS AND MAY WARRANT LOSS OF CREDIT!**  There are a total of 150 points in this homework. **Applies to Undergraduate Students Only:** You are graded out of 100 points meaning you only need 100 points to earn a 100% grade. You can earn more than 100 points for extra credit. There are 50 extra points each worth 0.2% of the notebook grade. Earning all 150 points yields a 110% grade. Given this assignment is worth 10% of your course grade, you can expect 1% extra course grade credit if you earn all 150 points. **Applies to Graduate Students Only:** You are graded out of 150 points. There is no extra credit for this assignment. These are the following gradable items and their weights

1. Statevectors (36 pts)
    A. Creating Statevectors from Dirac BraKet Notation (12 pts)
    B. Evolving Statevectors with Quantum Gates (12 pts)
    C. Multi-Qubit Statevectors (12 pts)
2. Building Gates (64 pts)
    A. Arbitrary Unitary Gates (20 pts)
    B. Gate Composition (24 pts)
    C. Gates from Quantum Circuits (20 pts)
3. The Swap Test (50 pts)
    A. Building a Swap Gate (10 pts)
    B. Building the Swap Test Circuit (10 pts)
    C. Turning the Swap Test into a Gate (5 pts)

D. Swap Test Experiments (25 pts)

<h3 id="Environment-Verification">Environment Verification</h3> Before you proceed, please verify your kernel is running the **Python 3.12** environment you created with `pip install -r requirements.txt`<br><br> If you don't see your environment in the kernel list, you need to add it using the terminal and relaunch jupyter. You can add your environment to interactive python using this command<br><br> `ipython kernel install --user --name=ENV_NAME` (replacing `ENV_NAME`)<br><br> After you've selected the correct environment, run this next cell, but do NOT edit it in any way! If no errors pop up, then your environment should be able to run this notebook with ease.

In [62]:
```
# DO NOT EDIT THIS CELL
n = 20

import sys
print("python", sys.version)

import numpy
print("numpy".ljust(n), numpy.__version__)

import qiskit
print("qiskit".ljust(n), qiskit.__version__)
```

```
python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD6
4)]
numpy                2.5.3
qiskit               2.5.2
```

<h3 id="Library-Imports">Library Imports</h3> We will need these libraries, classes, and utilities to write and run code for this homework. There should be no import errors if everything is installed correctly.

In [63]:
```
# General
import numpy as np

# Qiskit imports
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector, Pauli
from qiskit.primitives import StatevectorEstimator
from qiskit.circuit.library import IGate, SXGate, XGate, YGate, ZGate, SGate, TGate
                                   HGate, UnitaryGate, RYGate, CXGate, CZGate, SwapGat
from qiskit.visualization import plot_bloch_multivector, plot_state_city
```

<h1 id="Section-1:-Statevectors-(36-pts)">**Section 1: Statevectors (36 pts)**</h1> This section is dedicated to working with statevectors using Qiskit. Statevectors are essentially the math structure storing quantum information.

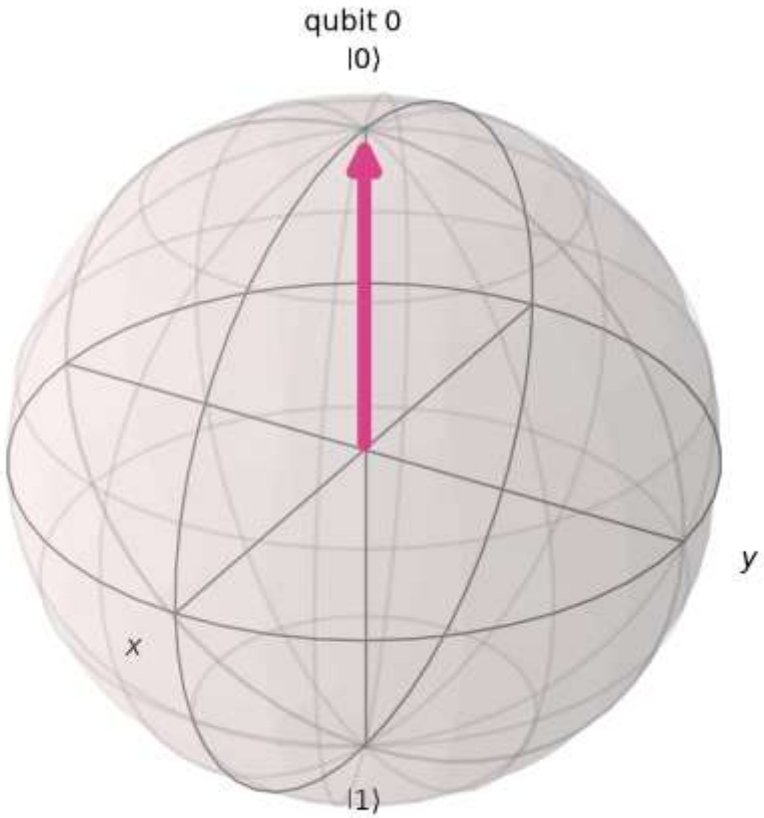
<h2 id="Part-1:-Creating-Statevectors-from-Dirac-BraKet-Notation-(12-pts)">**Part 1: Creating Statevectors from Dirac BraKet Notation (12 pts)**</h2> Using Qiskit's `Statevector` class, You will be constructing the statevectors for the single-qubit quantum

states $|0\rangle$, $|1\rangle$, $|+\rangle$, $|i\rangle$. You must verify your statevector appear correct using `plot_bloch_multivector`.

<h3 id="-Statevector-(3-pts)">$|0\rangle$ Statevector (3 pts)</h3> Create the `Statevector` for $|0\rangle$ and plot it.

In [6]:
```
state = Statevector.from_label('0') # creating the state |0>
plot_bloch_multivector(state) # plotting the state on bloch sphere
```

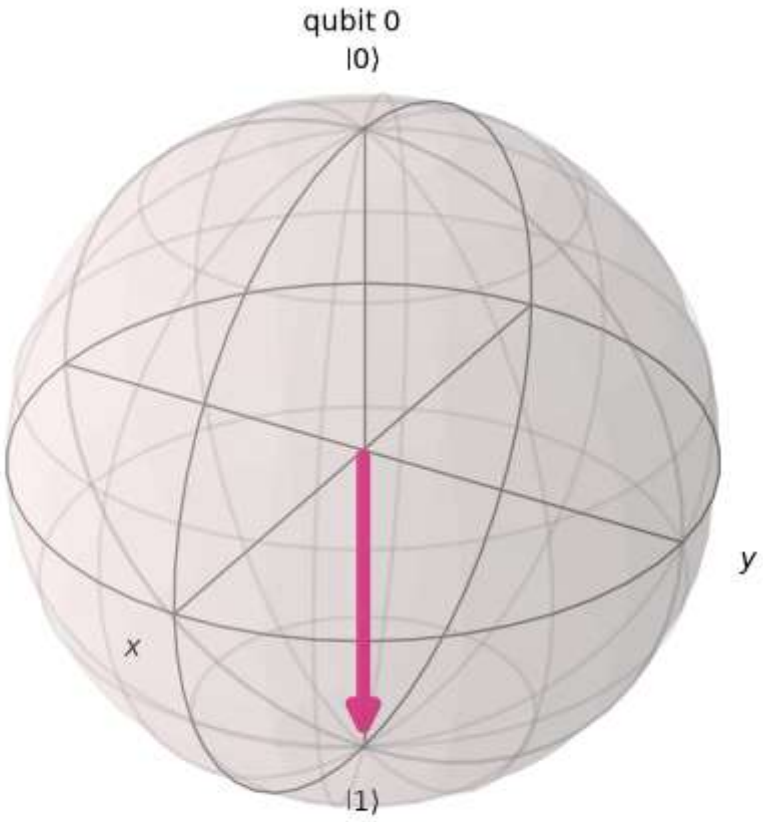
Out[6]:




<h3 id="-Statevector-(3-pts)">$|1\rangle$ Statevector (3 pts)</h3> Create the `Statevector` for $|1\rangle$ and plot it.

In [7]:
```
# YOUR CODE BELOW
# Create the Statevector for |1>
state_one = Statevector.from_label('1')

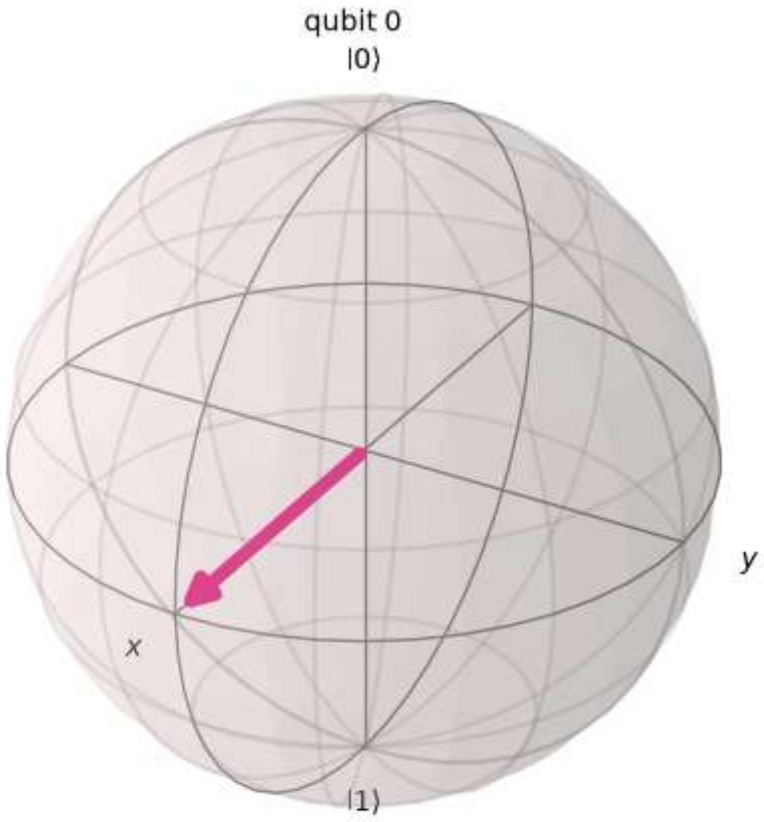
# Plot it on the Bloch sphere
display(plot_bloch_multivector(state_one))
```




<h3 id="-Statevector-(3-pts)">$|+\rangle$ Statevector (3 pts)</h3> Create the `Statevector` for $|+\rangle$ and plot it.

In [46]:
```
# YOUR CODE BELOW
state = Statevector.from_label('+') # creating the state |+>
plot_bloch_multivector(state) # plotting the state on bloch sphere
```

Out[46]:

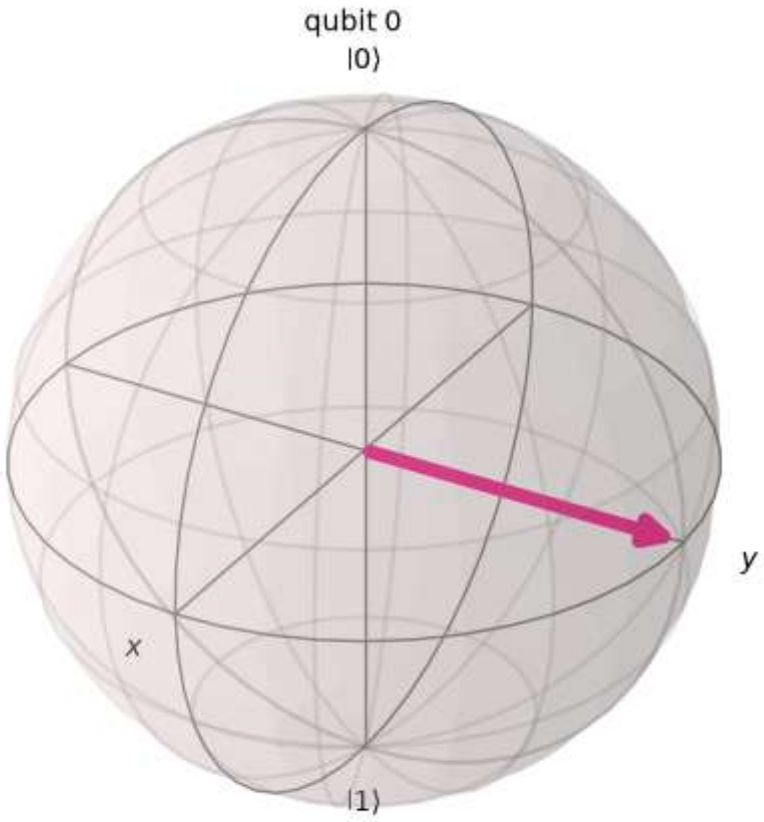



<h3 id="-Statevector-(3-pts)">$|i\rangle$ Statevector (3 pts)</h3> Create the `Statevector` for $|i\rangle$ and plot it.

In [47]:
```
# YOUR CODE BELOW

state = Statevector([1/np.sqrt(2), 1j/np.sqrt(2)]) # creating the state |i>
plot_bloch_multivector(state) # plotting the state on bloch sphere
```

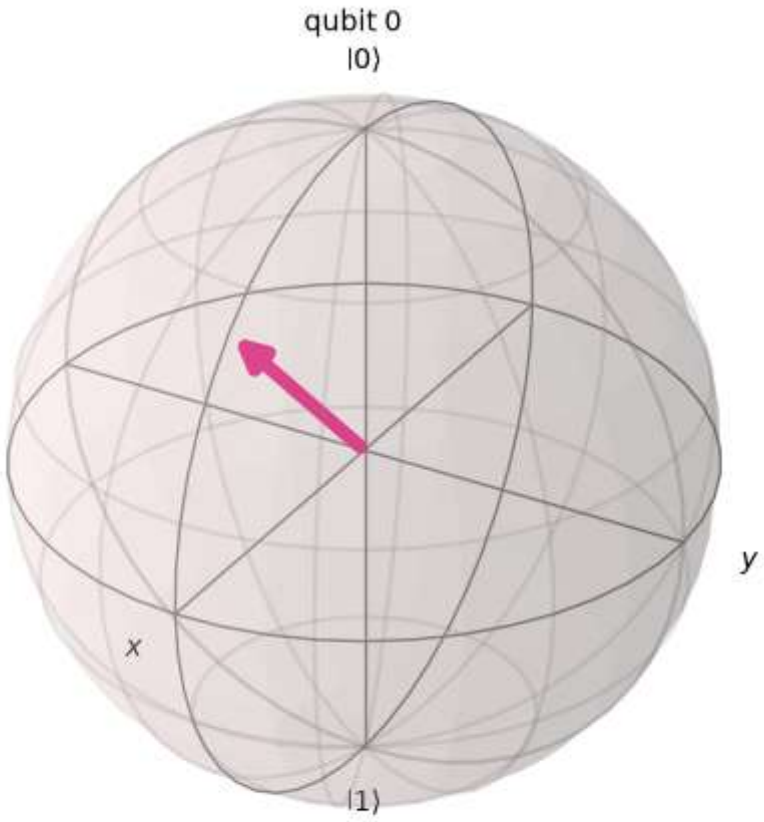
Out[47]:




<h2 id="Part-2:-Evolving-Statevectors-with-Quantum-Gates-(12-pts)">**Part 2: Evolving Statevectors with Quantum Gates (12 pts)**</h2> Given statevector `sv_initial`, You will be using its `evolve` method to apply a variety of gates from Qiskit's Standard Gates library and plot the results with `plot_bloch_multivector`.

In [64]:
```
sv_initial = Statevector(np.array([np.sin(3*np.pi/8), np.cos(3*np.pi/8)])) # initia
plot_bloch_multivector(sv_initial) # plotting the initial state on bloch sphere
```

Out[64]:

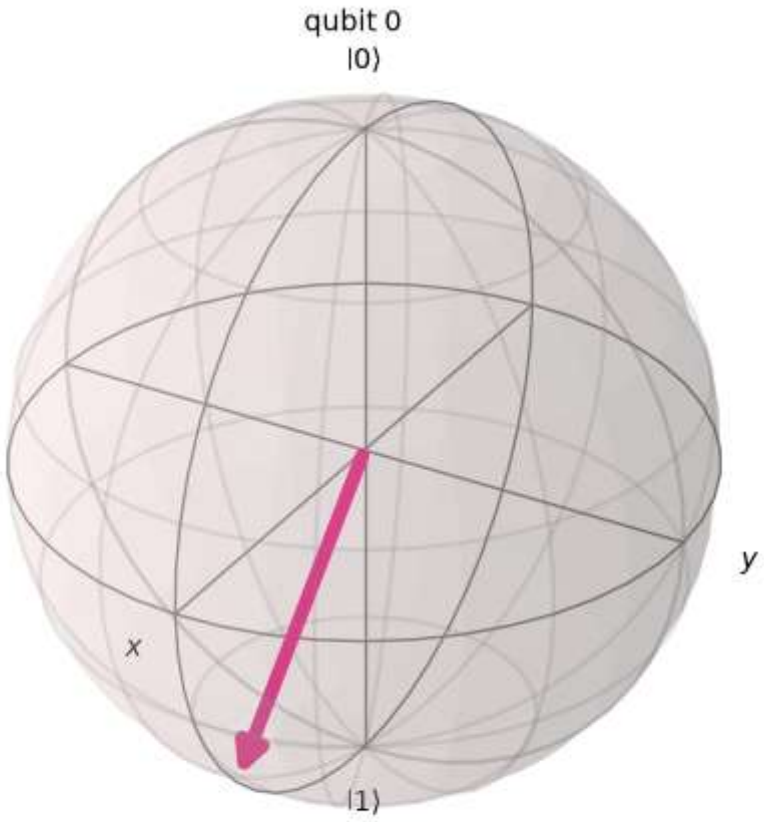



<h2 id="Evolve-using-XGate-(3-pts)">Evolve using XGate (3 pts)</h2> Apply an `XGate` to the statevector `sv_initial` using its `evolve` method and plot the result with `plot_bloch_multivector`.

In [48]:
```
# YOUR CODE BELOW

sv_initial = Statevector(np.array([np.sin(3*np.pi/8), np.cos(3*np.pi/8)])) # initia
sv_final = sv_initial.evolve(XGate()) # evolving the state with X gate
plot_bloch_multivector(sv_final) # plotting the final state on bloch sphere
```

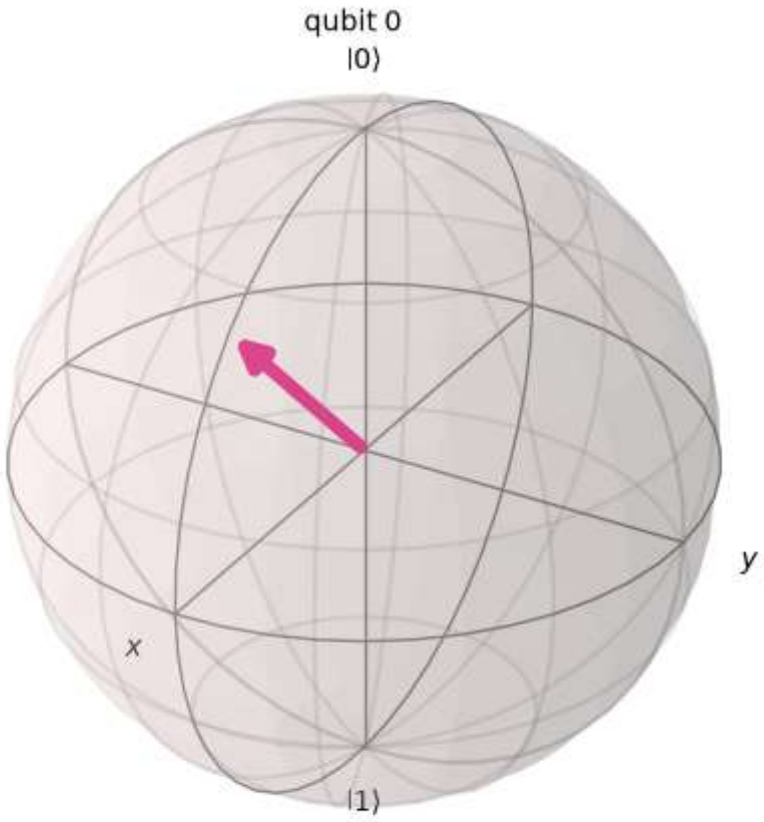
Out[48]:




<h2 id="Evolve-using-HGate-(3-pts)">Evolve using HGate (3 pts)</h2> Apply an `HGate` to the statevector `sv_initial` using its `evolve` method and plot the result with `plot_bloch_multivector`.

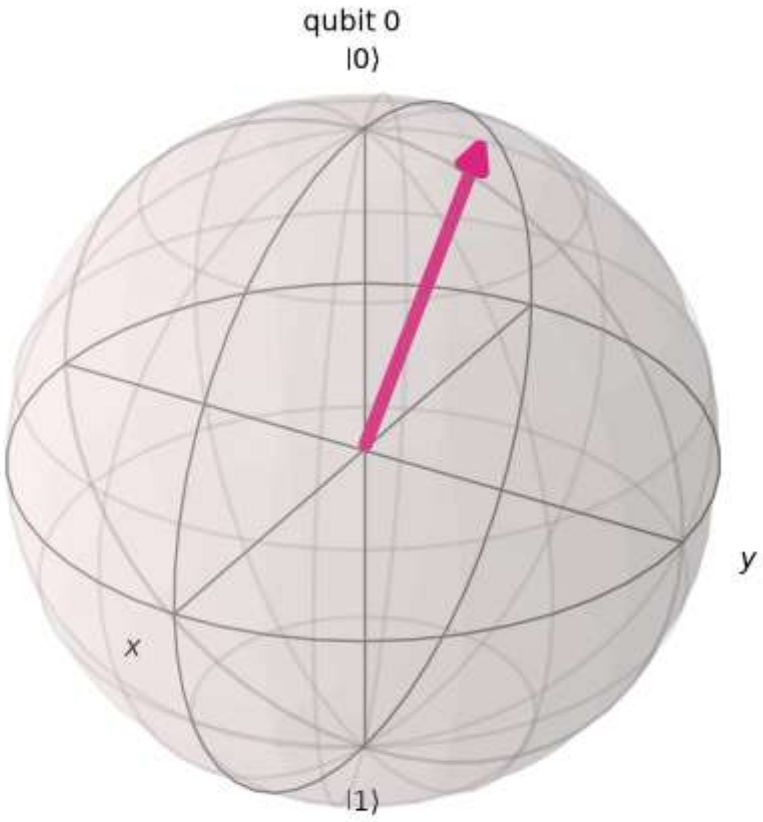
In [49]:
```
# YOUR CODE BELOW
sv_initial = Statevector(np.array([np.sin(3*np.pi/8), np.cos(3*np.pi/8)])) # initia
sv_final = sv_initial.evolve(HGate()) # evolving the state with H gate
plot_bloch_multivector(sv_final) # plotting the final state on bloch sphere
```

Out[49]:



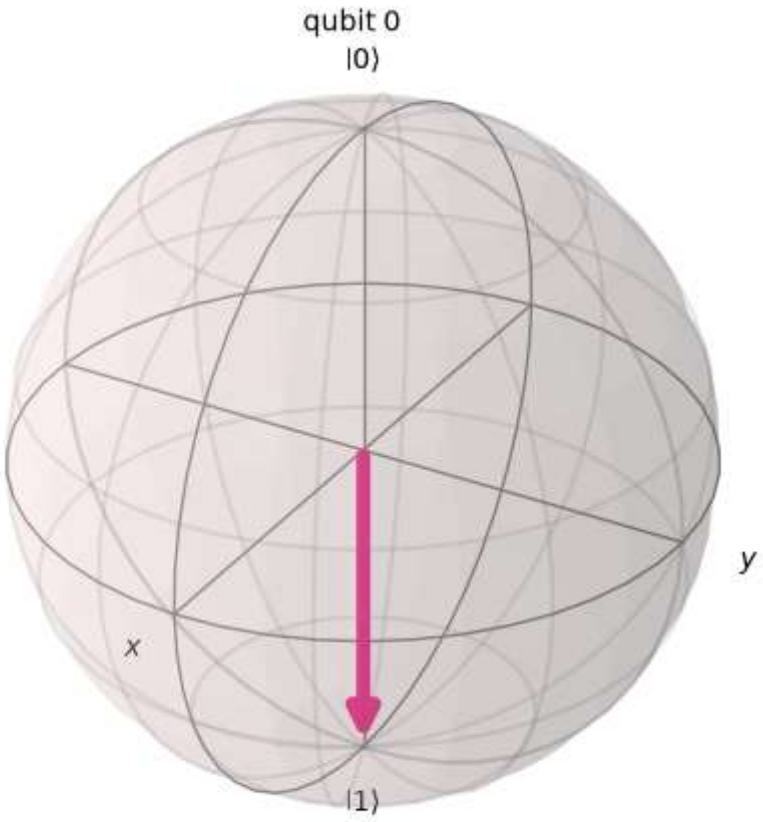

<h2 id="Evolve-using-ZGate-(3-pts)">Evolve using ZGate (3 pts)</h2> Apply a ZGate to the statevector sv_initial using its evolve method and plot the result with plot_bloch_multivector.

In [50]:
```
# YOUR CODE BELOW
sv_initial = Statevector(np.array([np.sin(3*np.pi/8), np.cos(3*np.pi/8)]))
sv_final = sv_initial.evolve(ZGate())
display(plot_bloch_multivector(sv_final))
```




<h2 id="Evolve-using-RYGate-(3-pts)">Evolve using RYGate (3 pts)</h2> Apply an `RYGate` with an angle of $\frac{3\pi}{4}$ to the statevector `sv_initial` using its `evolve` method and plot the result with `plot_bloch_multivector`.

In [51]:
```
# YOUR CODE BELOW
sv_initial = Statevector(np.array([np.sin(3*np.pi/8), np.cos(3*np.pi/8)]))
sv_final = sv_initial.evolve(RYGate(3 * np.pi / 4))
display(plot_bloch_multivector(sv_final))
```




<h2 id="Part-3:-Multi-Qubit-Statevectors-(12-pts)">**Part 3: Multi-Qubit Statevectors (12 pts)**</h2> Now that we understand how to construct single-qubit statevectors and apply single-qubit gates to them, we will work with multi-qubit statevectors.

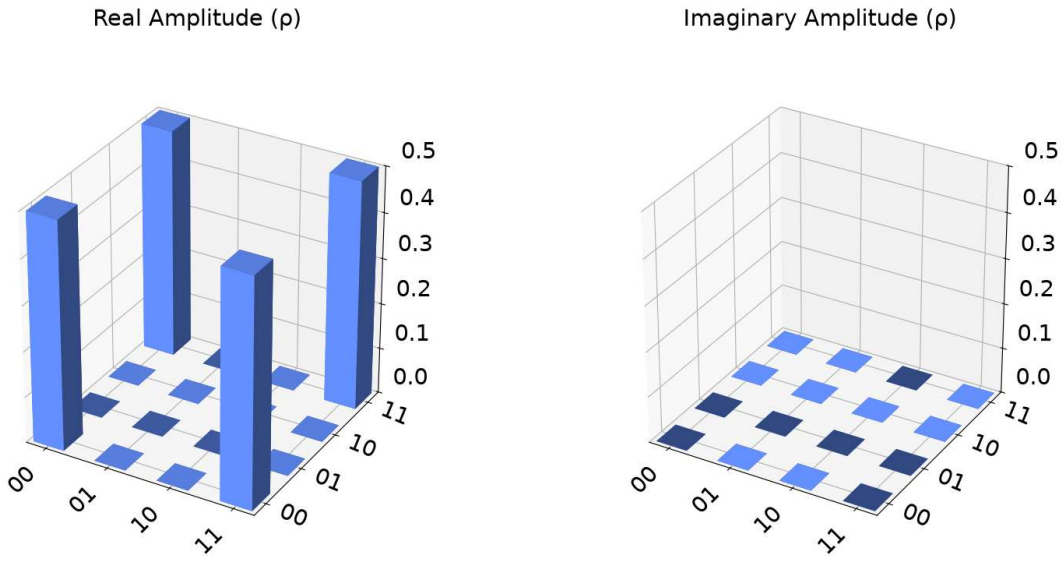
<h2 id="Creating-Entangled-States-(8-pts)">Creating Entangled States (8 pts)</h2>
There are 4 maximally entangled 2-qubit quantum states known as "Bell states". These are known as $|\Phi^+\rangle$, $|\Phi^-\rangle$, $|\Psi^+\rangle$, and $|\Psi^-\rangle$. You can find more info on them on Wikipedia. You will be creating a `Statevector` for each one and plotting it using `plot_state_city` 

<h3 id="-Statevector-(2-pts)">$|\Phi^+\rangle$ Statevector (2 pts)</h3> Create the `Statevector` for $|\Phi^+\rangle$ and plot it.

In [66]:
```
# YOUR CODE BELOW

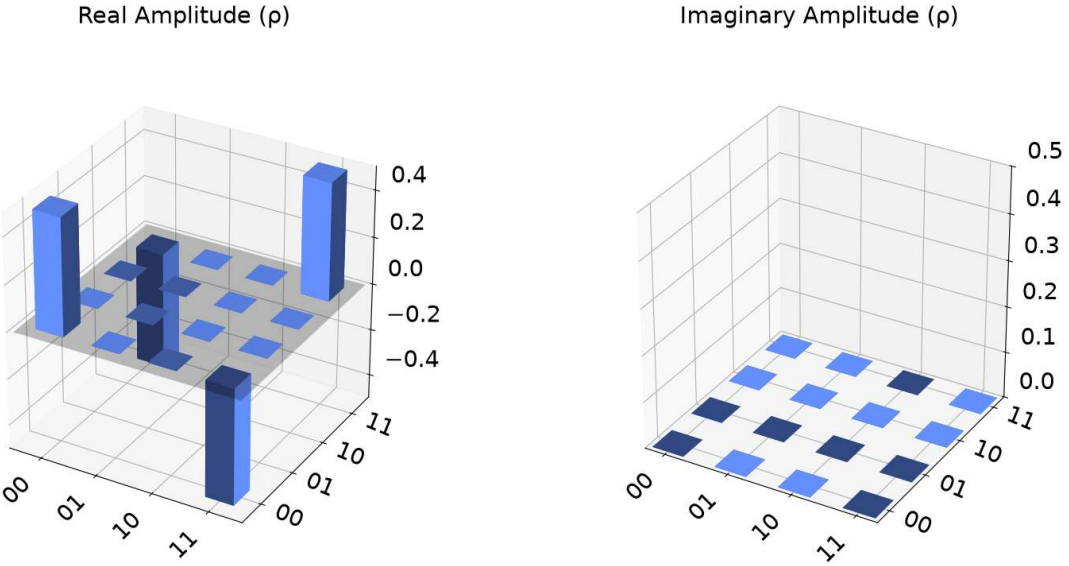
sv = Statevector.from_label('00') # initial state |00>
sv = sv.evolve(HGate(), [0]) # applying H gate on qubit 0
sv = sv.evolve(CXGate(), [0, 1]) # applying CX gate on qubits 0 and 1
plot_state_city(sv) # plotting the final state
```

Out[66]:




<h3 id="-Statevector-(2-pts)">$|\Phi^-\rangle$ Statevector (2 pts)</h3> Create the `Statevector` for $|\Phi^-\rangle$ and plot it.

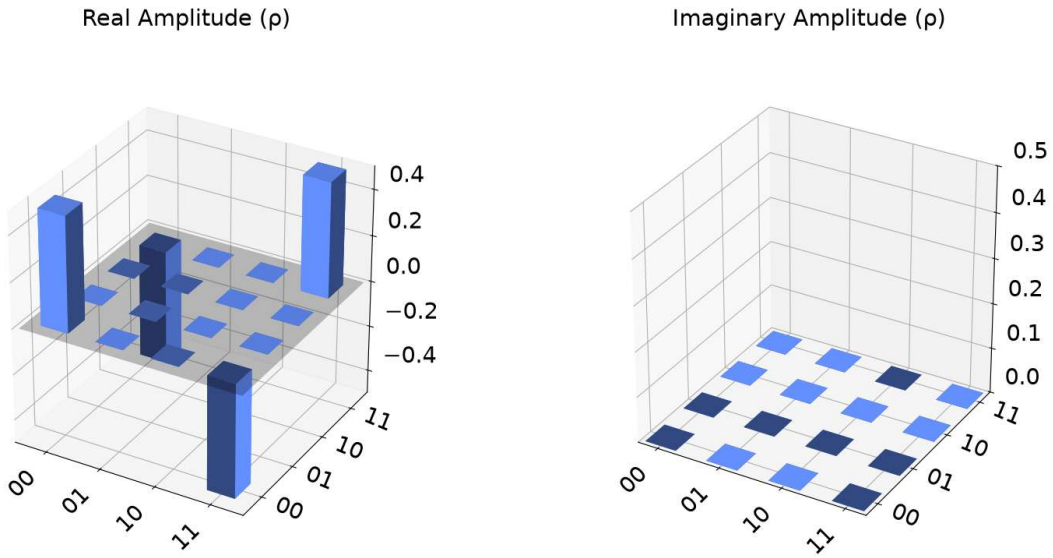
In [68]:
```
# YOUR CODE BELOW
phi_minus = Statevector([1/np.sqrt(2), 0, 0, -1/np.sqrt(2)])
display(plot_state_city(phi_minus))
```




<h3 id="-Statevector-(2-pts)">$|\Psi^+\rangle$ Statevector (2 pts)</h3> Create the `Statevector` for $|\Psi^+\rangle$ and plot it.

In [69]:
```
# YOUR CODE BELOW
sv = Statevector.from_label('01') # initial state |01>
sv = sv.evolve(HGate(), [0]) # applying H gate on qubit 0
sv = sv.evolve(CXGate(), [0, 1]) # applying CX gate on qubits 0 and 1
plot_state_city(sv) # plotting the final state
```

Out[69]:



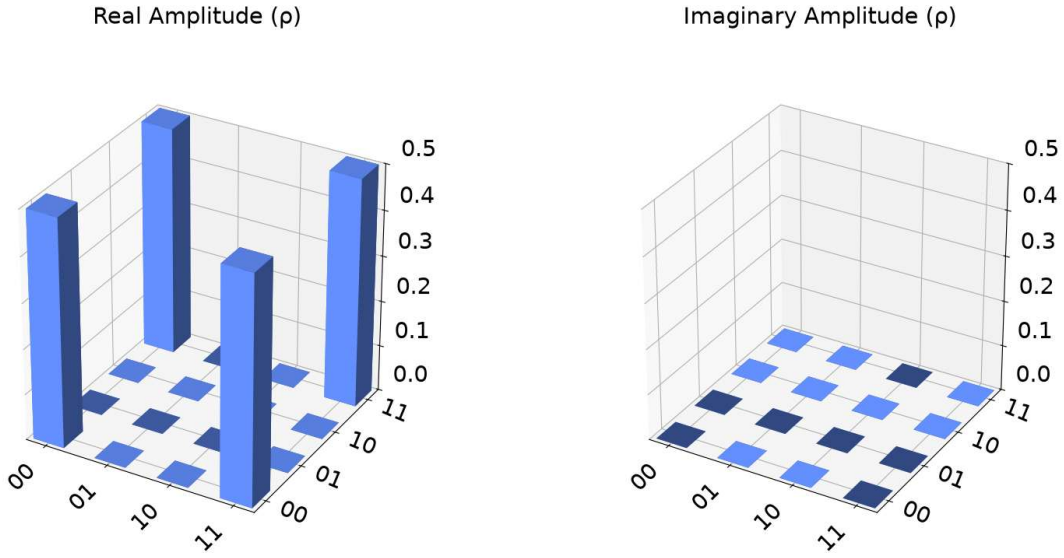

<h3 id="-Statevector-(2-pts)">$|\Psi^-\rangle$ Statevector (2 pts)</h3> Create the `Statevector` for $|\Psi^-\rangle$ and plot it.

In [70]:

```
# YOUR CODE BELOW

sv = Statevector.from_label('01') # initial state |01>
sv = sv.evolve(HGate(), [0]) # applying H gate on qubit 0
sv = sv.evolve(CXGate(), [0, 1]) # applying CX gate on qubits 0 and 1
sv = sv.evolve(ZGate(), [0]) # applying Z gate on qubit 0
plot_state_city(sv) # plotting the final state
```

Out[70]:

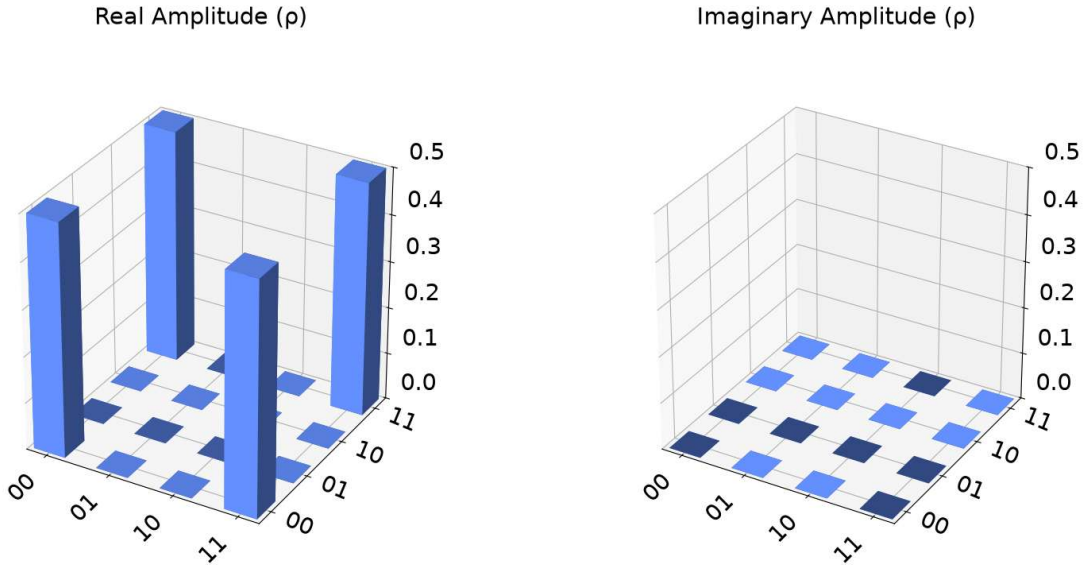



<h3 id="Operations-with-Multi-Qubit-Statevectors-(4-pts)">Operations with Multi-Qubit Statevectors (4 pts)</h3> Find and use a 2-qubit gate on the statevector $|\Phi^-\rangle$ to transform it to the statevector $|\Phi^+\rangle$ and plot it.

In [18]:

```
# YOUR CODE BELOW
phi_minus = Statevector([1/np.sqrt(2), 0, 0, -1/np.sqrt(2)])
```

```
phi_plus_transformed = phi_minus.evolve(ZGate(), [0])
display(plot_state_city(phi_plus_transformed))
```



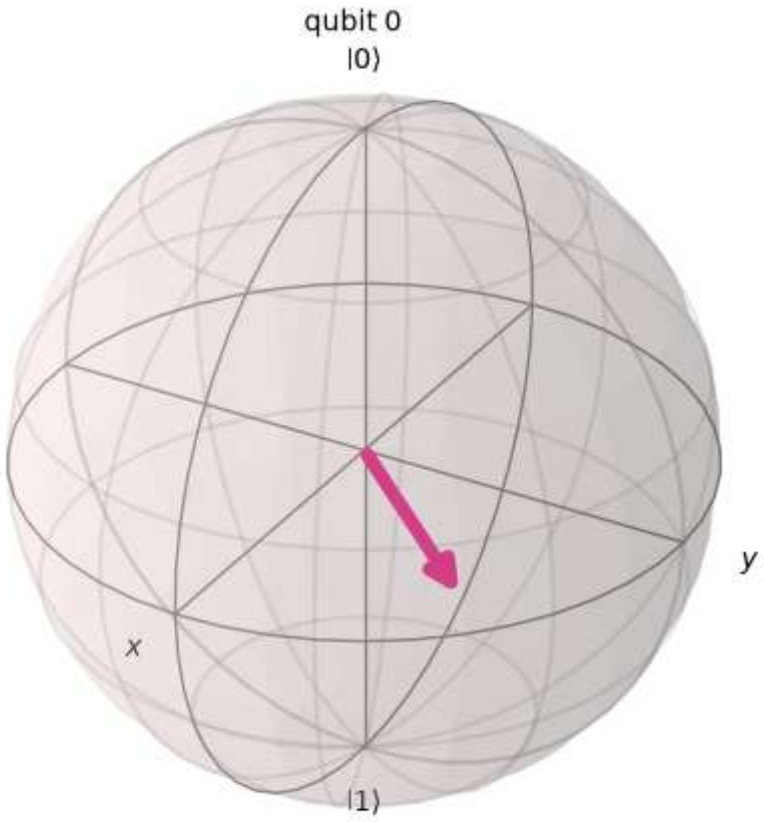

<h1 id="Section-2:-Building-Gates-(64-pts)">**Section 2: Building Gates (64 pts)**</h1>
This next section is dedicated to building custom gates by matrices, composition of gates, and quantum circuits.

<h2 id="Part-1:-Arbitrary-Unitary-Gates-(20-pts)">**Part 1: Arbitrary Unitary Gates (20 pts)**</h2> Using `UnitaryGate`, you will create custom unitary gates from unitary matrices.

<h3 id="An-Example-(4-pts)">An Example (4 pts)</h3> Use the given matrix `arbitrary_unitary_matrix` and create a `UnitaryGate` with it. Then `evolve` the statevector $|0\rangle$ with it. Finally plot the resulting statevector using `plot_bloch_multivector`.

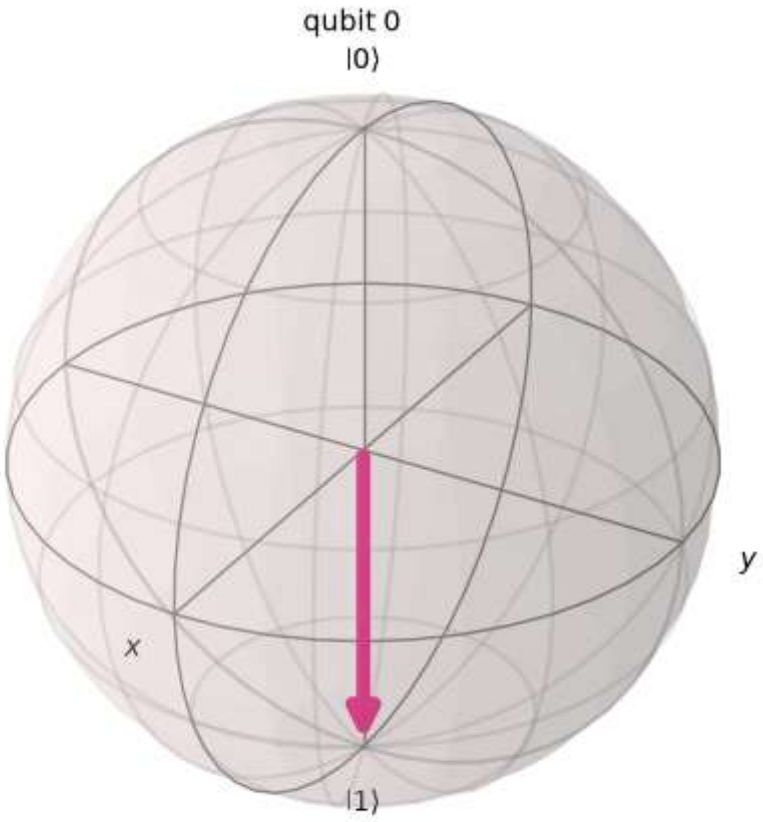
In [52]:
```
theta = 2*np.pi/5
arbitrary_unitary_gate = np.array([
    [np.cos(theta),np.sin(theta)],
    [-np.sin(theta),np.cos(theta)]
])
# YOUR CODE BELOW
sv = Statevector.from_label('0') # initial state |0>
sv = sv.evolve(UnitaryGate(arbitrary_unitary_gate)) # evolving the state with the a
plot_bloch_multivector(sv) # plotting the final state
```

Out[52]:



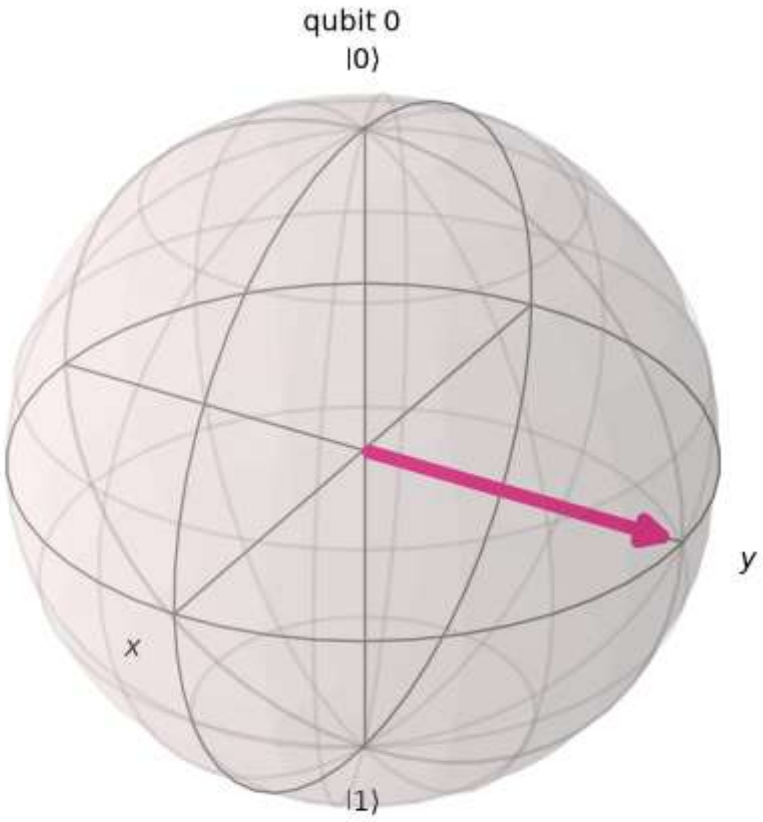

<h3 id="Creating-an-$X$-Gate-(4-pts)">Creating an $X$ Gate (4 pts)</h3> Design a numpy array that performs a $\pi$-rotation around the X-axis. Create a `UnitaryGate` with that matrix. Then `evolve` the statevector $|0\rangle$ with it. Finally plot the resulting statevector using `plot_bloch_multivector`.

In [23]:
```
x_matrix = np.array([[0, 1], [1, 0]])
x_gate = UnitaryGate(x_matrix)
sv_initial = Statevector.from_label('0')
sv_final = sv_initial.evolve(x_gate)
display(plot_bloch_multivector(sv_final))
```



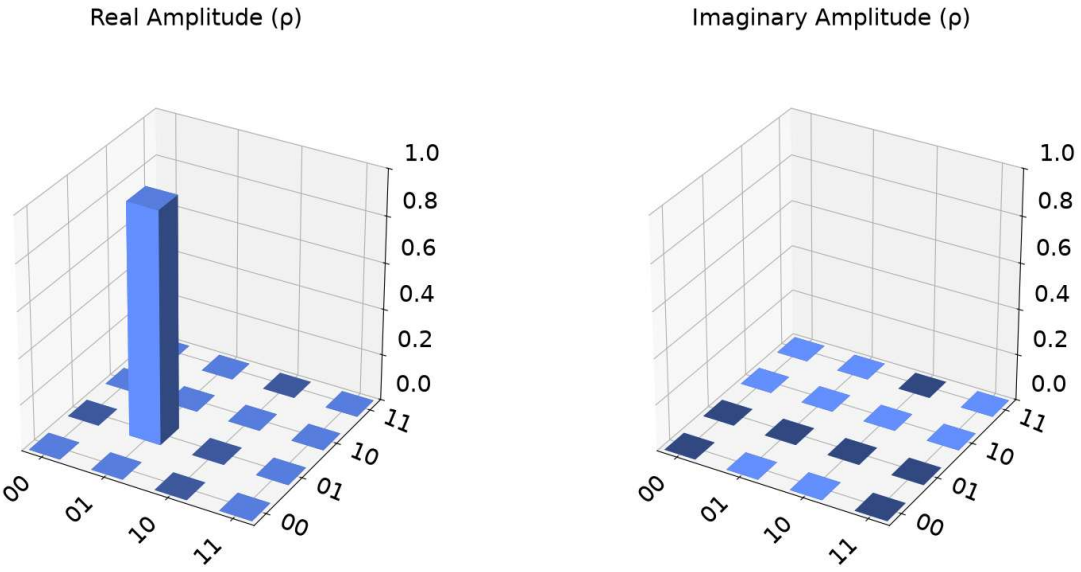

<h3 id="Creating-an-$S$-Gate-(4-pts)">Creating an $S$ Gate (4 pts)</h3> Design a numpy array that performs a $\frac{\pi}{2}$-rotation around the Z-axis. Create a `UnitaryGate` with that matrix. Then `evolve` the statevector $|+\rangle$ with it. Finally plot the resulting statevector using `plot_bloch_multivector`.

In [24]:
```
# YOUR CODE BELOW
s_matrix = np.array([[1, 0], [0, 1j]])
s_gate = UnitaryGate(s_matrix)
sv_initial = Statevector.from_label('+')
sv_final = sv_initial.evolve(s_gate)
display(plot_bloch_multivector(sv_final))
```




<h3 id="Creating-a-CNOT-Gate-(8-pts)">Creating a CNOT Gate (8 pts)</h3> Design a numpy array that performs a controlled-NOT gate. Create a `UnitaryGate` with that matrix. Then `evolve` the statevector $|01\rangle$ into $|11\rangle$ using that gate. Finally plot the resulting statevector using `plot_state_city`.

In [25]:
```
# YOUR CODE BELOW
cnot_matrix = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 1],
    [0, 0, 1, 0]
])
cnot_gate = UnitaryGate(cnot_matrix)
sv_initial = Statevector.from_label('01')
sv_final = sv_initial.evolve(cnot_gate)
display(plot_state_city(sv_final))
```




<h2 id="Part-2:-Gate-Composition-(24-pts)">**Part 2: Gate Composition (24 pts)**</h2>
Using numpy's `matmul` you will construct new quantum gates that perform a series of basis gates in one operation.

<h3 id="Gate-Composition-Function-(12-pts)">Gate Composition Function (12 pts)</h3>
Finish the `compose_single_qubit_gates` python function such that it constructs a gate starting from identity through matrix multiplication (`np.matmul`) using the each gate's matrix (`to_matrix()`), then turns the final matrix into a `UnitaryGate`.

In [26]:
```
def compose_single_qubit_gates(*gates) -> UnitaryGate:
    # Notice on ordering:
    # Suppose *gates is XGate(), HGate()
    # then the composition is to apply a
    # rotation equivalent to an XGate() rotation,
    # then an HGate() rotation.

    # YOUR CODE BELOW
    matrix = np.eye(2)
    for gate in gates:
        matrix = np.matmul(gate.to_matrix(), matrix)
    return UnitaryGate(matrix)
```

<h3 id="The-$T$-Gate-(4-pts)">The $T^\dagger$ Gate (4 pts)</h3> Create a custom gate that is equivalent to the `TdgGate` by using only the `ZGate`, `SGate`, and `TGate`, then store it in the `gate_tdg` variable.

In [27]:
```
# YOUR CODE ABOVE
gate_tdg = compose_single_qubit_gates(ZGate(), SGate(), TGate())

# This checks if the gates perform the same rotation.
# If no error, then your answer is correct.
assert np.allclose(gate_tdg.to_matrix(), TdgGate().to_matrix())
```

<h3 id="The-$X$-Gate-(4-pts)">The $X$ Gate (4 pts)</h3> Create a custom gate that is equivalent to the XGate by using only the ZGate and HGate, then store it in the gate_x variable.

In [28]:
```
# YOUR CODE ABOVE
gate_x = compose_single_qubit_gates(HGate(), ZGate(), HGate())

# This checks if the gates perform the same rotation.
# If no error, then your answer is correct.
assert np.allclose(gate_x.to_matrix(), XGate().to_matrix())
```

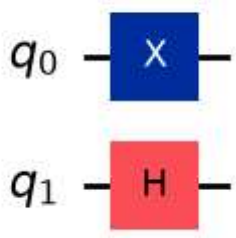
<h3 id="The-$S$-Gate-(4-pts)">The $S$ Gate (4 pts)</h3> Create a custom gate that is equivalent to the SGate by using only the HGate and SXGate, then store it in the gate_s variable.

In [29]:
```
# YOUR CODE ABOVE
gate_s = compose_single_qubit_gates(TGate(), TGate())
# This checks if the gates perform the same rotation.
# If no error, then your answer is correct.
assert np.allclose(gate_s.to_matrix(), SGate().to_matrix())
```

<h2 id="Part-3:-Gates-from-Quantum-Circuits-(20-pts)">**Part 3: Gates from Quantum Circuits (20 pts)**</h2> You will create `QuantumCircuit` objects that perform a selection of gates on some qubits, then turn that circuit into its own new gate operation to be used in other circuits.

<h3 id="Building-a-Quantum-Circuit-(8-pts)">Building a Quantum Circuit (8 pts)</h3> Create a `QuantumCircuit` with 2 qubits stored it in a variable qc that performs an $X$-gate on the first qubit and a $H$-gate on the second qubit. Then use its draw method in the matplotlib style.

In [30]:
```
# YOUR CODE BELOW
qc = QuantumCircuit(2)
qc.x(0)
qc.h(1)
display(qc.draw('mpl'))
```



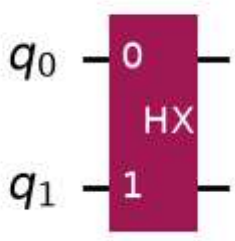

<h3 id="Converting-the-Circuit-to-a-Gate-(4-pts)">Converting the Circuit to a Gate (4 pts)</h3> Using the quantum circuit qc from the previous cell, convert it into a quantum gate gate_qc with a `label` $HX$ using its `to_gate` method.

In [31]:
```
# YOUR CODE BELOW
gate_qc = qc.to_gate(label='HX')
```

<h3 id="Using-the-Circuit-Gate-(4-pts)">Using the Circuit-Gate (4 pts)</h3> Create a new 2-qubit quantum circuit qc_gate_applied and using its append method, add the gate gate_qc to it. Draw the new circuit in matplotlib style.

In [53]:
```
# YOUR CODE BELOW
qc_gate_applied = QuantumCircuit(2) # creating a quantum circuit with 2 qubits
qc_gate_applied.append(gate_qc, [0, 1]) # applying the gate on qubits 0 and 1
qc_gate_applied.draw('mpl') # drawing the quantum circuit
```

Out[53]:




<h3 id="Checking-Equivalence-(2-pts)">Checking Equivalence (2 pts)</h3> Check if your circuits qc and qc_gate_applied are equivalent.

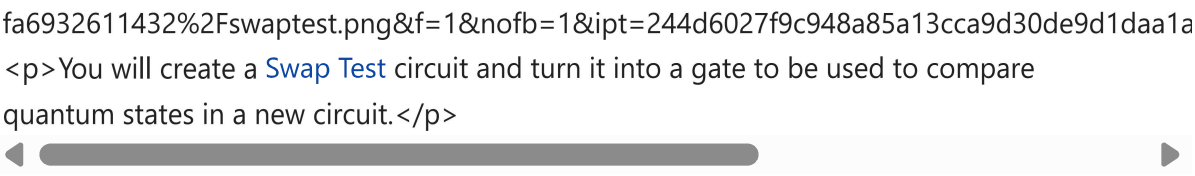
In [56]:
```
qc == qc_gate_applied
```

Out[56]:
```
False
```

<h3 id="Establishing-Equivalence-with-Decomposition-(2-pts)">Establishing Equivalence with Decomposition (2 pts)</h3> You should see despite qc and qc_gate_applied performing the same basis gates on the same qubits, their construction differs which is why they are not equivalent in Qiskit, but there's a way to check for basis gate equivalence. decompose the HX gate in qc_gate_applied to yield a new quantum circuit and compare that to qc.

In [57]:
```
qc_gate_decomposed = qc_gate_applied.decompose() # decomposing the quantum circuit
qc == qc_gate_decomposed # checking if the original quantum circuit and the decompo
```

Out[57]:
```
True
```

<h1 id="Section-3:-The-Swap-Test-(50-pts)">**Section 3: The Swap Test (50 pts)**</h1>
**This section is intended for graduate students, but may be completed by undergraduate students for extra credit** 

You will create a Swap Test circuit and turn it into a gate to be used to compare quantum states in a new circuit.

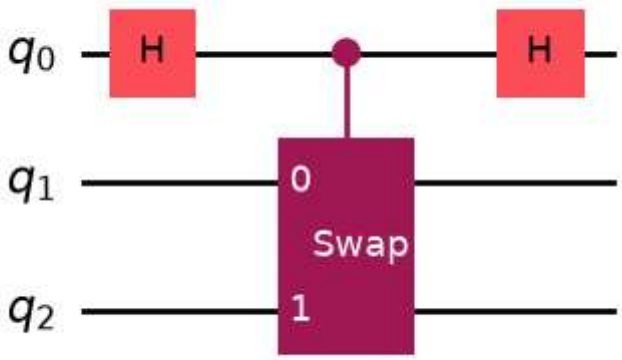
<h2 id="Part-1:-Building-a-Swap-Gate-(10-pts)">**Part 1: Building a Swap Gate (10 pts)**</h2> Design a 4x4 numpy array that swaps the quantum states of two qubits, then create a `UnitaryGate` with a `label` *Swap* from the matrix and store it in a variable `gate_swap`. Do not use `SwapGate`.

In [34]:
```
# YOUR CODE ABOVE
swap_matrix = np.array([
    [1, 0, 0, 0],
    [0, 0, 1, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 1]
])
gate_swap = UnitaryGate(swap_matrix, label='Swap')

# Ensures your swap gate performs the same as the SwapGate
assert np.allclose(SwapGate().to_matrix(), gate_swap.to_matrix())
```

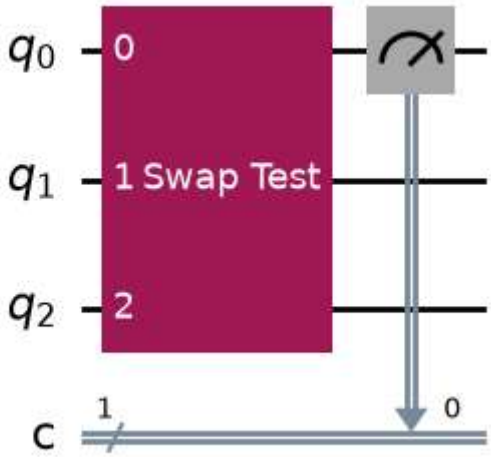
<h2 id="Part-2:-Building-the-Swap-Test-Circuit-(10-pts)">**Part 2: Building the Swap Test Circuit (10 pts)**</h2> Create a 3-qubit quantum circuit stored in `qc_swap_test`. Then apply a hadamard gate on the first qubit, then apply your `gate_swap` targeting the second and third qubits controlled by the first. Finally, apply another hadamard gate on the first qubit, but *do not measure it*. Draw the circuit using the matplotlib style.

In [35]:
```
# YOUR CODE BELOW
qc_swap_test = QuantumCircuit(3)
qc_swap_test.h(0)
qc_swap_test.append(gate_swap.control(), [0, 1, 2])
qc_swap_test.h(0)
display(qc_swap_test.draw('mpl'))
```




<h2 id="Part-3:-Turning-the-Swap-Test-into-a-Gate-(5-pts)">**Part 3: Turning the Swap Test into a Gate (5 pts)**</h2> Convert the Swap Test circuit `qc_swap_test` into a quantum gate `gate_swap_test` with a `label` *Swap Test*. Apply it to a new 3-qubit 1-classical bit quantum circuit `qc_swap_draw`. Apply a measurement to the first qubit, then draw this circuit with matplotlib style.

In [36]:
```
# YOUR CODE BELOW
gate_swap_test = qc_swap_test.to_gate(label='Swap Test')
qc_swap_draw = QuantumCircuit(3, 1)
qc_swap_draw.append(gate_swap_test, [0, 1, 2])
qc_swap_draw.measure(0, 0)
display(qc_swap_draw.draw('mpl'))
```




<h2 id="Part-4:-Swap-Test-Experiments-(25-pts)">**Part 4: Swap Test Experiments (25 pts)**</h2> You will perform the swap test on a variety of single-qubit quantum state pairs to understand how the similarity measure works.

<h3 id="Circuit-Simulation-Function-(10-pts)">Circuit Simulation Function (10 pts)</h3> Finish the code for the `swap_test_expectation_value` Python function. Using the `StatevectorEstimator`, run a primitive unified block (PUB) containing `circuit` and an

observable equivalent to measuring the first qubit in the computational basis using `Pauli`. Use the estimator's `run` on the pub enclosed in a list. Then fetch the `result` and grab its `data.evs` to return a similarity measure value.

In [58]:
```
def swap_test_expectation_value(circuit: QuantumCircuit) -> float:

    estimator = StatevectorEstimator() # creating an instance of StatevectorEstimat
    observable = Pauli('ZII')  # observable for measuring the first qubit in the Z
    pub = (circuit, observable) # preparing the input for the estimator
    result = estimator.run([pub]).result() # running the estimator
    final = result[0].data.evs  # extracting the expectation value
    return float(final) # returning the expectation value as a float
```

<h3 id="Swap-Test-on--and--(3-pts)">Swap Test on $|0\rangle$ and $|1\rangle$ (3 pts)</h3> Create a quantum circuit `qc_swap_test_10` with 3 qubits. Initialize the second qubit in $|0\rangle$ and the third qubit in $|1\rangle$. Apply the swap test gate `gate_swap_test` to the appropriate qubits. Print the Swap Test result using `swap_test_expectation_value`. Draw the circuit in matplotlib style.

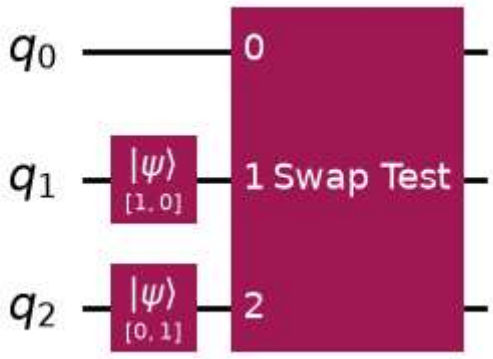
In [71]:
```
# YOUR CODE BELOW

qc_swap_test = QuantumCircuit(3) # creating a 3-qubit quantum circuit
# Applying to |0> and |1>
qc_swap_test_10 = QuantumCircuit(3)
qc_swap_test_10.initialize([1, 0], 1)  # |0>
qc_swap_test_10.initialize([0, 1], 2)  # |1>
qc_swap_test_10.append(gate_swap_test, [0, 1, 2])

print("Swap Test Expectation Value (|0>, |1>):", swap_test_expectation_value(qc_swa
qc_swap_test_10.draw('mpl')
```

```
Swap Test Expectation Value (|0>, |1>): -5.551115123125783e-17
```

Out[71]:




<h3 id="Swap-Test-on--and--(3-pts)">Swap Test on $|0\rangle$ and $|+\rangle$ (3 pts)</h3> Create a quantum circuit `qc_swap_test_plus0` with 3 qubits. Initialize the second qubit in $|0\rangle$ and the third qubit in $|+\rangle$. Apply the swap test gate `gate_swap_test` to the appropriate qubits. Print the Swap Test result using `swap_test_expectation_value`. Draw the circuit in matplotlib style.

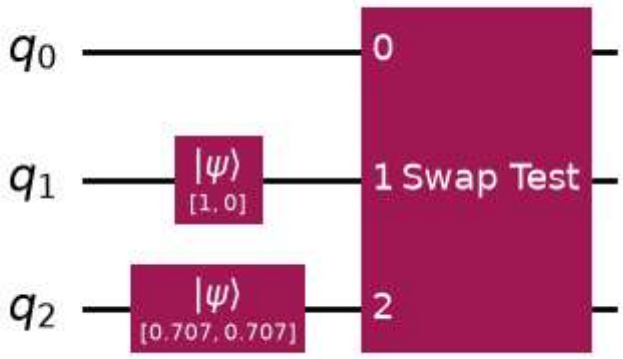
In [60]:
```
qc_swap_test_plus0 = QuantumCircuit(3)
qc_swap_test_plus0.initialize([1, 0], 1)  # Initialize second qubit to |0>
qc_swap_test_plus0.initialize([1/np.sqrt(2), 1/np.sqrt(2)], 2)  # Initialize third

qc_swap_test_plus0.append(gate_swap_test, [0, 1, 2])

print("Swap Test Expectation Value (|0>, |+>):", swap_test_expectation_value(qc_swa
qc_swap_test_plus0.draw('mpl')
```

```
Swap Test Expectation Value (|0>, |+>): 0.49999999999999994
```

Out[60]:



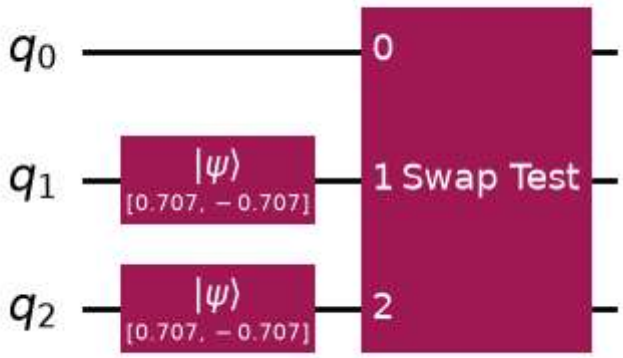

<h3 id="Swap-Test-on--and--(3-pts)">Swap Test on $|-\rangle$ and $|-\rangle$ (3 pts)</h3> Create a quantum circuit `qc_swap_test_minusminus` with 3 qubits. Initialize the second qubit in $|-\rangle$ and the third qubit in $|-\rangle$. Apply the swap test gate `gate_swap_test` to the appropriate qubits. Print the Swap Test result using `swap_test_expectation_value`. Draw the circuit in matplotlib style.

In [72]:
```
# YOUR CODE BELOW

qc_swap_test_minusminus = QuantumCircuit(3)
qc_swap_test_minusminus.initialize([1/np.sqrt(2), -1/np.sqrt(2)], 1)  # Initialize
qc_swap_test_minusminus.initialize([1/np.sqrt(2), -1/np.sqrt(2)], 2)  # Initialize
qc_swap_test_minusminus.append(gate_swap_test, [0, 1, 2])
print("Swap Test Expectation Value (|->, |->):", swap_test_expectation_value(qc_swa
qc_swap_test_minusminus.draw('mpl')
```

```
Swap Test Expectation Value (|->, |->): 6.661338147750939e-16
```

Out[72]:

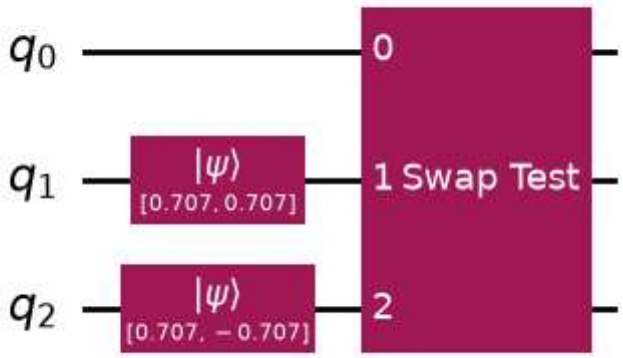



<h3 id="Swap-Test-on--and--(3-pts)">Swap Test on $|-\rangle$ and $|+\rangle$ (3 pts)</h3> Create a quantum circuit `qc_swap_test_plusminus` with 3 qubits. Initialize the second qubit in $|-\rangle$ and the third qubit in $|+\rangle$. Apply the swap test gate `gate_swap_test` to the appropriate qubits. Print the Swap Test result using `swap_test_expectation_value`. Draw the circuit in matplotlib style.

In [73]:
```
# YOUR CODE BELOW
qc_swap_test_plusminus = QuantumCircuit(3)
qc_swap_test_plusminus.initialize([1/np.sqrt(2), 1/np.sqrt(2)], 1)  # Initialize se
qc_swap_test_plusminus.initialize([1/np.sqrt(2), -1/np.sqrt(2)], 2)  # Initialize t
qc_swap_test_plusminus.append(gate_swap_test, [0, 1, 2]) # applying the swap test g
print("Swap Test Expectation Value (|+>, |->):", swap_test_expectation_value(qc_swa
qc_swap_test_plusminus.draw('mpl') # drawing the quantum circuit
```

```
Swap Test Expectation Value (|+>, |->): -6.106226635438361e-16
```

Out[73]:




<h3 id="Swap-Test-on-Near-Identical-States-(3-pts)">Swap Test on Near-Identical States (3 pts)</h3> Create a quantum circuit `qc_swap_test_plusminus` with 3 qubits. Initialize the second and third qubits in $|1\rangle$, but apply a $R_x(\frac{\pi}{8})$-gate to the third qubit. Apply the swap test gate `gate_swap_test` to the appropriate qubits. Print the Swap Test result using `swap_test_expectation_value`. Draw the circuit in matplotlib style.

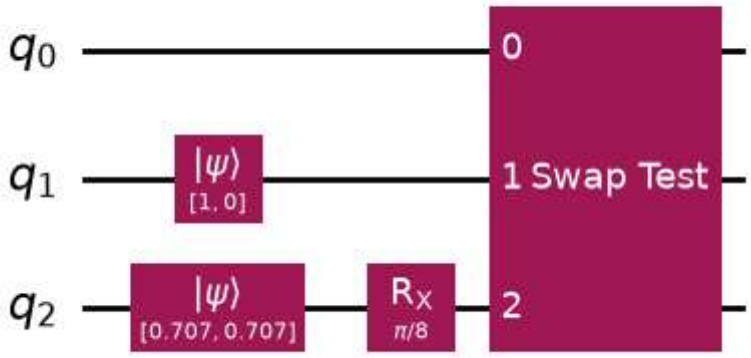
In [74]:
```
# YOUR CODE BELOW

qc_swap_test_plusminus = QuantumCircuit(3)
```

```
qc_swap_test_plusminus.initialize([1, 0], 1)  # Initialize second qubit to |1>
qc_swap_test_plusminus.initialize([1/np.sqrt(2), 1/np.sqrt(2)], 2)  # Initialize th
qc_swap_test_plusminus.rx(np.pi/8, 2)  # Apply Rx(pi/8) gate to third qubit
qc_swap_test_plusminus.append(gate_swap_test, [0, 1, 2])
print("Swap Test Expectation Value (|1>, |+>):", swap_test_expectation_value(qc_swa
qc_swap_test_plusminus.draw('mpl')
```

```
Swap Test Expectation Value (|1>, |+>): 0.49999999999999967
```

Out[74]:




<h1 id="Submitting-Your-Work">Submitting Your Work</h1> If you have completed this programming homework exercise, well done! <br><br> To submit your work, please follow these steps.

1. Convert this notebook to HTML by opening a terminal, and running<br><br> `jupyter nbconvert --to html FILEPATH_TO_NOTEBOOK` (replacing `FILEPATH_TO_NOTEBOOK`)<br><br>
2. Open your HTML file in a browser
3. Right-click in the browser and click print
4. Change print settings to save as a pdf, set the margin to none, and downscale (if necessary, but don't make it too small)
5. Upload your PDF file to the Pilot dropbox